VECTOR

MICROSAR CryIf

Technical Reference

Crypto Interface

Version 5.0.0

Authors	Markus Schneider, Philipp Ritter, Mathias Waldenburger, Max Engelhardt
Status	Released

Document Information

History

Author	Date	Version	Remarks
Schneider, Markus	2017-03-07	1.01.00	Initial creation of Technical Reference
Ritter, Philipp	2017-05-08	1.02.00	Changed chapter 5.1.6, 5.1.7, 5.1.8, 5.1.12
Waldenburger, Mathias	2019-03-15	4.01.00	KeyElementCopyPartial, APIs added
Engelhardt, Max	2019-12-11	5.00.00	Chapter 3.1: Support for asynchronous key handling Chapter 6.2: add chapter 6.2.4

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_CryptoInterface.pdf	4.3.0
[2]	AUTOSAR	AUTOSAR_SWS_DET.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_SWS_CryptoInterface.pdf	4.4.0

Contents

1 Component History 6

2 Introduction 7
2.1 Architecture Overview 7

3 Functional Description 9
3.1 Features 9
3.1.1 KeyElementCopyPartial 9
3.2 Initialization 9
3.3 States 10
3.4 Main Functions 10
3.5 Error Handling 10
3.5.1 Development Error Reporting 10

4 Integration 12
4.1 Scope of Delivery 12
4.1.1 Static Files 12
4.1.2 Dynamic Files 12

5 API Description 13
5.1 Services provided by CRYIF 13
5.1.1 CryIf_InitMemory 13
5.1.2 CryIf_Init 13
5.1.3 CryIf_GetVersionInfo 14
5.1.4 CryIf_ProcessJob 14
5.1.5 CryIf_CancelJob 15
5.1.6 CryIf_KeyElementSet 16
5.1.7 CryIf_KeySetValid 16
5.1.8 CryIf_KeyElementGet 17
5.1.9 CryIf_KeyElementCopy 18
5.1.10 CryIf_KeyElementCopyPartial 18
5.1.11 CryIf_KeyCopy 20
5.1.12 CryIf_RandomSeed 20
5.1.13 CryIf_KeyGenerate 21
5.1.14 CryIf_KeyDerive 21
5.1.15 CryIf_KeyExchangeCalcPubVal 22
5.1.16 CryIf_KeyExchangeCalcSecret 23
5.1.17 CryIf_CertificateParse 24
5.1.18 CryIf_CertificateVerify 24

© 2019 Vector Informatik GmbH

5.2 Services used by CRYIF 25
5.3 Callback Functions 25
5.3.1 CryIf_CallbackNotification 25

6 Configuration 26
6.1 Configuration Variants 26
6.2 Configuration with DaVinci Configurator 5 Pro 26
6.2.1 General Properties 26
6.2.2 Channel Properties 26
6.2.3 Key Properties 27
6.2.4 Crypto Module 27

7 Glossary and Abbreviations 29
7.1 Glossary 29
7.2 Abbreviations 29

8 Contact 30

Illustrations

Figure 2-1 AUTOSAR 4.3 Architecture Overview 7
Figure 2-2 Interfaces to adjacent modules of the CRYIF 8

Tables

Table 1-1 Component history 6
Table 3-1 Supported AUTOSAR standard conform features according to [1] 9
Table 3-2 Supported AUTOSAR standard conform features according to [3] 9
Table 3-3 Service IDs 10
Table 3-4 Errors reported to DET 11
Table 4-1 Static files 12
Table 4-2 Generated files 12
Table 5-1 CryIf_InitMemory 13
Table 5-2 CryIf_Init 14
Table 5-3 CryIf_GetVersionInfo 14
Table 5-4 CryIf_ProcessJob 15
Table 5-5 CryIf_CancelJob 15
Table 5-6 CryIf_KeyElementSet 16
Table 5-7 CryIf_KeySetValid 17
Table 5-8 CryIf_KeyElementGet 17
Table 5-9 CryIf_KeyElementCopy 18
Table 5-10 CryIf_KeyElementCopyPartial 19
Table 5-11 CryIf_KeyCopy 20
Table 5-12 CryIf_RandomSeed 21
Table 5-13 CryIf_KeyGenerate 21
Table 5-14 CryIf_KeyDerive 22
Table 5-15 CryIf_KeyExchangeCalcPubVal 23
Table 5-16 CryIf_KeyExchangeCalcSecret 23
Table 5-17 CryIf_CertificateParse 24
Table 5-18 CryIf_CertificateVerify 25
Table 5-19 Services used by the CRYIF 25
Table 5-20 CryIf_CallbackNotification 25
Table 6-1 General Properties 26
Table 6-2 Channel Properties 26
Table 6-3 Key Properties 27
Table 6-4 Crypto Module 28
Table 7-1 Glossary 29
Table 7-2 Abbreviations 29

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial beta release
1.01.00	Adaptions to the specification; several improvements and bug fixes
1.02.00	Release of component
4.00.00	Support redirection of input/output buffers from/to key elements Support partial key element copy
4.01.00	Release of component
5.00.00	Introduce support for asynchronous key handling

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CRYIF as specified in [1].

Supported AUTOSAR Release*:	4.3	
Supported Configuration Variants:	pre-compile	
Vendor ID:	CRYIF_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CRYIF_MODULE_ID	112 decimal (according to [1])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The Crypto Interface (CRYIF) is called by the Cryptographic Service Manager (CSM) to forward its service requests to the underlying Crypto Drivers (CRYPTO). The CRYIF has access to the CRYPTO to calculate results with their cryptographic services. These results are returned to the CSM by the CRYIF.

2.1 Architecture Overview

The following figure shows where the CRYIF is located in the AUTOSAR architecture.

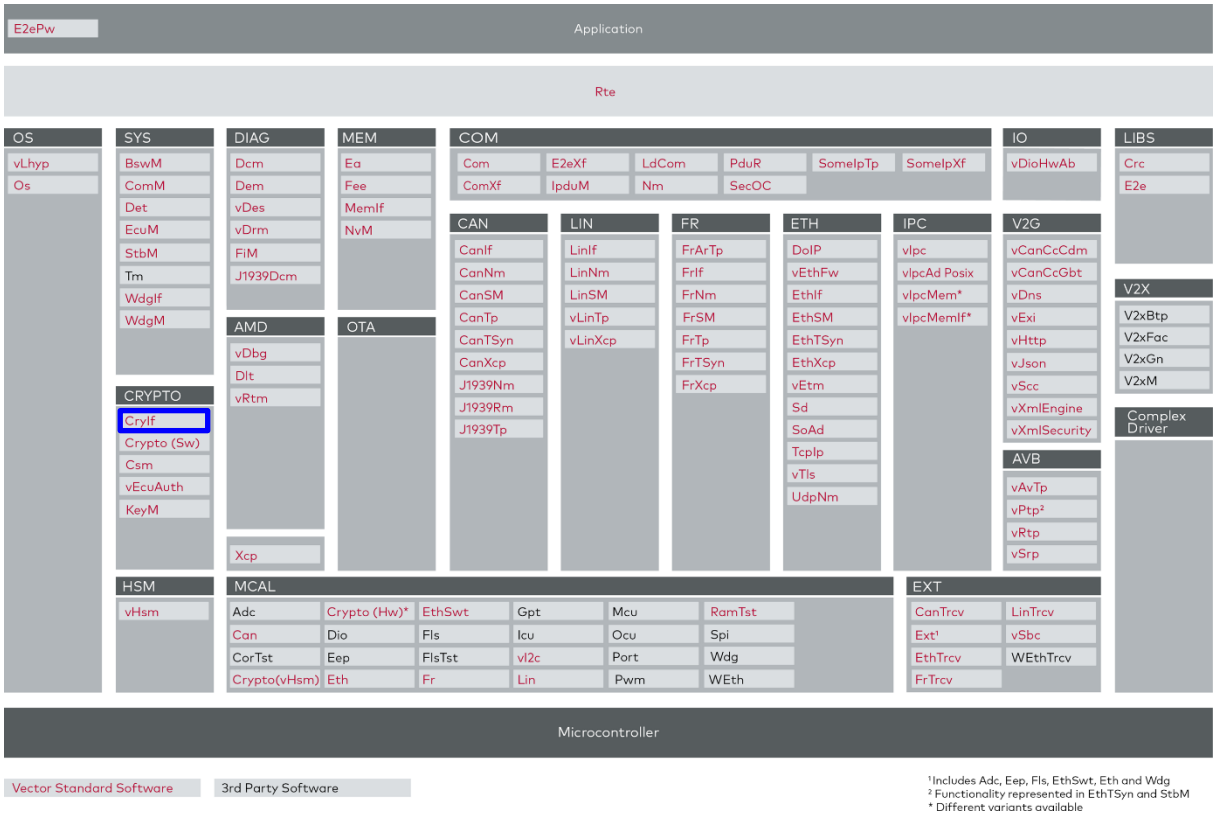


Figure 2-1 AUTOSAR 4.3 Architecture Overview

The next figure shows the interfaces to adjacent modules of the CRYIF. These interfaces are described in chapter 5.

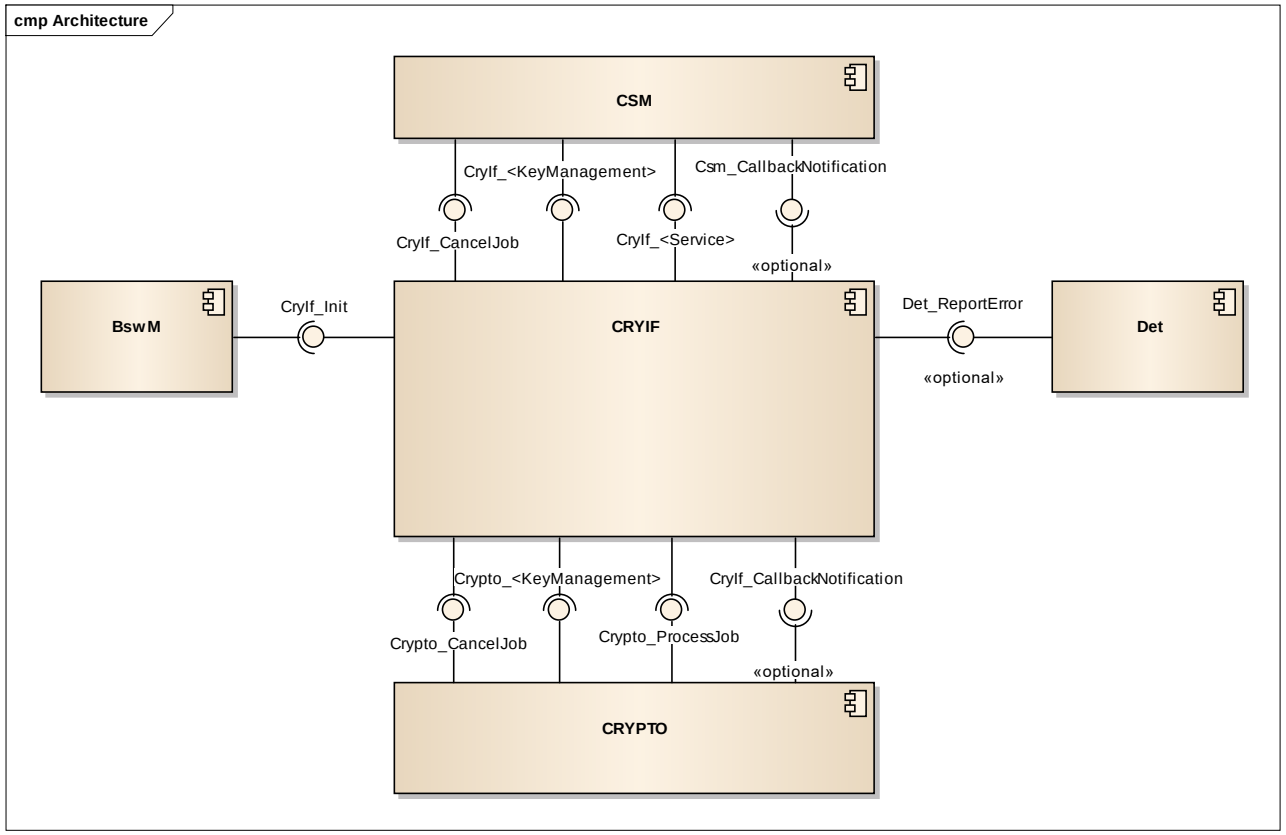


Figure 2-2 Interfaces to adjacent modules of the CRYIF

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the CRYIF.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Dispatching jobs to the configured Crypto Driver
Dispatching key management functionalities
Forward Callback Notification

Table 3-1 Supported AUTOSAR standard conform features according to [1]

The following features specified in [3] are supported:

Supported AUTOSAR Standard Conform Features
Redirection of input and output buffers to key elements
KeyElementCopyPartial API
Processing of asynchronous key management jobs

Table 3-2 Supported AUTOSAR standard conform features according to [3]

3.1.1 KeyElementCopyPartial

If the underlying Crypto driver does not support the Crypto_KeyElementCopyPartial API, the CryIf handles partial copying by itself. The copying of the CryIf also takes place if the cryIfKeyId and the targetCryIfKeyId point to different Crypto drivers.

First, the procedure gets the contents of both key elements via Crypto_KeyElementGet API. Afterwards, the partial copying takes place by copying the requested part of the source key element to the requested location in the target key element. Last, the constructed key element is written to the Crypto driver via Crypto_KeyElementSet API.

If the target offset is larger than the length of the target key element, a zero-padding is applied in between.

3.2 Initialization

Before any other functionality of the CRYIF module can be called the initialization function `CryIf_Init()` must be called by the BSWM.

For manual null initialization of RAM variables, the CRYIF offers the function `CryIf_InitMemory()` which can be called before the `CryIf_Init()`.

3.3 States

The CRYIF does not have a state machine.

3.4 Main Functions

CRYIF does not provide a main function. All calls are synchronous.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `CRYIF_DEV_ERROR_REPORT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but it must have the same signature as the service `Det_ReportError()`.

The reported CRYIF ID is 112.

The reported service IDs identify the services which are described in 5.1. The following table presents the service IDs and the related services:

Service ID	Service
0x00	CryIf_Init
0x01	CryIf_GetVersionInfo
0x02	CryIf_ProcessJob
0x03	CryIf_CancelJob
0x04	CryIf_KeyElementSet
0x05	CryIf_KeySetValid
0x06	CryIf_KeyElementGet
0x0f	CryIf_KeyElementCopy
0x10	CryIf_KeyCopy
0x07	CryIf_RandomSeed
0x08	CryIf_KeyGenerate
0x09	CryIf_KeyDerive
0x0A	CryIf_KeyExchangeCalcPubVal
0x0B	CryIf_KeyExchangeCalcSecret
0x0C	CryIf_CertificateParse
0x11	CryIf_CertificateVerify
0x12	CryIf_KeyElementCopyPartial

Table 3-3 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	API service used without module initialization
0x01	Initialization of CRYIF module failed
0x02	API request called with invalid parameter (null pointer)
0x03	API request called with invalid parameter (out of range)
0x04	API request called with invalid parameter (invalid value)
0x11	The service CryIf_Init() is called while the module is already initialized

Table 3-4 Errors reported to DET

4 Integration

This chapter gives necessary information for the integration of the MICROSAR CRYIF into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the CRYIF contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
CryIf.c	This file contains the CRYIF source code.
CryIf.h	This is the header file of the CRYIF.
CryIf_Cbk.h	This is the callback header file of CRYIF.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator 5 Pro

File Name	Description
CryIf_Cfg.c	This is configuration source file.
CryIf_Cfg.h	This is configuration header file.

Table 4-2 Generated files

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Services provided by CRYIF

5.1.1 CryIf_InitMemory

Prototype	
`void CryIf_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Power-up memory initialization.	
Particularities and Limitations	
Use this function in case these variables are not initialized by the startup code. Module is uninitialized. Initialize component variables at power up.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-1 CryIf_InitMemory

5.1.2 CryIf_Init

Prototype	
`void CryIf_Init (void)`	
Parameter	
ConfigPtr [in]	Configuration structure for initializing the module
Return code	
void	none
Functional Description	
Initialization function.	
Particularities and Limitations	
Specification of module initialization > Interrupts are disabled. Module is uninitialized. CryIf_InitMemory has been called unless CryIf_ModuleInitialized is initialized by start-up code.	

This function initializes the module CryIf. It initializes all variables and sets the module state to Initialized.
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 5-2 CryIf_Init

5.1.3 CryIf_GetVersionInfo

Prototype	
`void CryIf_GetVersionInfo (Std_VersionInfoType *versioninfo)`	
Parameter	
versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
none CryIf_GetVersionInfo() returns version information, vendor ID and AUTOSAR module ID of the component.	
Call context	
> TASK|ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-3 CryIf_GetVersionInfo

5.1.4 CryIf_ProcessJob

Prototype	
`Std_ReturnType CryIf_ProcessJob (uint32 channelId, Crypto_JobType *job)`	
Parameter	
channelId [in]	Holds the identifier of the crypto channel.
job [in,out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.

	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_NOT_VALID Request failed, the key is not valid.
	CRYPTO_E_QUEUE_FULL Request failed, the queue is full.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Process the received job.	
Particularities and Limitations	
none This interface dispatches the received jobs to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-4 CryIf_ProcessJob

5.1.5 CryIf_CancelJob

Prototype	
`Std_ReturnType` **`CryIf_CancelJob`** `(uint32 channelId, Crypto_JobType *job)`	
Parameter	
channelId [in]	Holds the identifier of the crypto channel.
job [in,out]	Pointer to the configuration of the job. Contains structures with user and primitive relevant information.
Return code	
Std_ReturnType	E_OK Request successful, job has been removed.
Std_ReturnType	E_NOT_OK Request failed, job could not be removed.
Functional Description	
Cancels the received job.	
Particularities and Limitations	
none This interface removes the provided job from the underlying Crypto Driver Object queue.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-5 CryIf_CancelJob

5.1.6 CryIf_KeyElementSet

Prototype	
`Std_ReturnType` **`CryIf_KeyElementSet`** `(uint32 cryIfKeyId, uint32 keyElementId, const uint8 *keyPtr, uint32 keyLength)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
keyPtr [in]	Holds the pointer to the key data which shall be set as key element.
keyLength [in]	Contains the length of the key element in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element size does not match size of provided data.
Functional Description	
Sets a key element.	
Particularities and Limitations	
none This function shall dispatch the key element set function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-6 CryIf_KeyElementSet

5.1.7 CryIf_KeySetValid

Prototype	
`Std_ReturnType` **`CryIf_KeySetValid`** `(uint32 cryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key elements shall be set to valid.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.

Functional Description
Sets the key to valid.
Particularities and Limitations
none This function shall dispatch the key set valid function to the configured crypto driver object.
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-7 CryIf_KeySetValid

5.1.8 CryIf_KeyElementGet

Prototype	
`Std_ReturnType` **`CryIf_KeyElementGet`** `(uint32 cryIfKeyId, uint32 keyElementId, uint8 *resultPtr, uint32 *resultLengthPtr)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key element shall be set.
keyElementId [in]	Holds the identifier of the key element which shall be set.
resultPtr [in]	Holds the pointer to the result data which shall be set as key element.
resultLengthPtr [in]	Contains the length of the result in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Exports the key element	
Particularities and Limitations	
none This function shall dispatch the get key element function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-8 CryIf_KeyElementGet

5.1.9 CryIf_KeyElementCopy

Prototype	
`Std_ReturnType` **`CryIf_KeyElementCopy`** `(uint32 cryIfKeyId, uint32 keyElementId, uint32 targetCryIfKeyId, uint32 targetKeyElementId)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
keyElementId [in]	Holds the identifier of the key element which shall be the source for the copy operation.
targetCryIfKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
targetKeyElementId [in]	Holds the identifier of the key element which shall be the destination for the copy operation.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_EXTRACT_DENIED Request failed, not allowed to extract key material.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
Functional Description	
Copy key element.	
Particularities and Limitations	
none This function shall copy a key element from one key to a target key.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-9 CryIf_KeyElementCopy

5.1.10 CryIf_KeyElementCopyPartial

Prototype
`Std_ReturnType` **`CryIf_KeyElementCopyPartial`** `(uint32 cryIfKeyId, uint32 keyElementId, uint32 keyElementSourceOffset, uint32 keyElementTargetOffset, uint32 keyElementCopyLength, uint32 targetCryIfKeyId, uint32 targetKeyElementId)`

Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
keyElementId [in]	Holds the identifier of the key element which shall be the source for the copy operation.
keyElementSourceOffset [in]	This is the offset of the source key element indicating the start index of the copy operation.
keyElementTargetOffset [in]	This is the offset of the destination key element indicating the start index of the copy operation.
keyElementCopyLength [in]	Specifies the number of bytes that shall be copied.
targetCryIfKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
targetKeyElementId [in]	Holds the identifier of the key element which shall be the destination for the copy operation.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
	CRYPTO_E_KEY_EMPTY Request failed because of uninitialized source key element.
Functional Description	
Copies a key element partially.	
Particularities and Limitations	
none This function copies a key element partially from one key to a target key.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-10 CryIf_KeyElementCopyPartial

5.1.11 CryIf_KeyCopy

Prototype	
`Std_ReturnType` **`CryIf_KeyCopy`** `(uint32 cryIfKeyId, uint32 targetCryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key whose key element shall be the source element.
targetCryIfKeyId [in]	Holds the identifier of the key whose key element shall be the destination element.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_KEY_READ_FAIL Request failed, read access was denied.
	CRYPTO_E_KEY_WRITE_FAIL Request failed, write access was denied.
	CRYPTO_E_KEY_NOT_AVAILABLE Request failed, the key is not available.
	CRYPTO_E_KEY_SIZE_MISMATCH Request failed, the key element sizes are not compatible.
Functional Description	
Copy the key.	
Particularities and Limitations	
none This function shall copy all key elements from the source key to a target key.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-11 CryIf_KeyCopy

5.1.12 CryIf_RandomSeed

Prototype	
`Std_ReturnType` **`CryIf_RandomSeed`** `(uint32 cryIfKeyId, const uint8 *seedPtr, uint32 seedLength)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key for which a new material shall be generated.
seedPtr [in]	Holds a pointer to the memory location which contains the data to feed the seed.
seedLength [in]	Contains the length of the seed in bytes.
Return code	
Std_ReturnType	E_OK Request successful.

	E_NOT_OK Request failed.
Functional Description	
Initialize the seed.	
Particularities and Limitations	
none This function shall dispatch the random seed function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-12 CryIf_RandomSeed

5.1.13 CryIf_KeyGenerate

Prototype	
`Std_ReturnType` **`CryIf_KeyGenerate`** `(uint32 cryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key which is to be updated with the generated value.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Generates a key.	
Particularities and Limitations	
none This function shall dispatch the key generate function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-13 CryIf_KeyGenerate

5.1.14 CryIf_KeyDerive

Prototype
`Std_ReturnType` **`CryIf_KeyDerive`** `(uint32 cryIfKeyId, uint32 targetCryIfKeyId)`

Parameter	
cryIfKeyId [in]	Holds the identifier of the key which is used for key derivation.
targetCryIfKeyId [in]	Holds the identifier of the key which is used to store the derived key.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Derives a key.	
Particularities and Limitations	
none This function shall dispatch the key derive function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-14 CryIf_KeyDerive

5.1.15 CryIf_KeyExchangeCalcPubVal

Prototype	
`Std_ReturnType` **`CryIf_KeyExchangeCalcPubVal`** `(uint32 cryIfKeyId, uint8 *publicValuePtr, uint32 *publicValueLengthPtr)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
publicValuePtr [out]	Contains the pointer to the data where the public value shall be stored.
publicValueLengthPtr [in,out]	Holds a pointer to the memory location in which the public value length information is stored. On calling this function, this parameter shall contain the size of the buffer provided by publicValuePtr. When the request has finished, the actual length of the returned value shall be stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the public value.	

Particularities and Limitations
none This function shall dispatch the key exchange public value calculation function to the configured crypto driver object.
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-15 CryIf_KeyExchangeCalcPubVal

5.1.16 CryIf_KeyExchangeCalcSecret

Prototype	
`Std_ReturnType` **`CryIf_KeyExchangeCalcSecret`** `(uint32 cryIfKeyId, const uint8 *partnerPublicValuePtr, uint32 partnerPublicValueLength)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key which shall be used for the key exchange protocol.
partnerPublicValuePtr [in]	Holds the pointer to the memory location which contains the partner's public value.
partnerPublicValueLength [in]	Contains the length of the partner's public value in bytes.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
	CRYPTO_E_SMALL_BUFFER Request failed, the provided buffer is too small to store the result.
Functional Description	
Calculation of the secret.	
Particularities and Limitations	
none This function shall dispatch the key exchange common shared secret calculation function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-16 CryIf_KeyExchangeCalcSecret

5.1.17 CryIf_CertificateParse

Prototype	
`Std_ReturnType` **`CryIf_CertificateParse`** `(uint32 cryIfKeyId)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key slot in which the certificate has been stored.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Parse stored certificate.	
Particularities and Limitations	
none This function shall dispatch the certificate parse function to the configured crypto driver object.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-17 CryIf_CertificateParse

5.1.18 CryIf_CertificateVerify

Prototype	
`Std_ReturnType` **`CryIf_CertificateVerify`** `(uint32 cryIfKeyId, uint32 verifyCryIfKeyId, Crypto_VerifyResultType *verifyPtr)`	
Parameter	
cryIfKeyId [in]	Holds the identifier of the key which shall be used to validate the certificate.
verifyCryIfKeyId [in]	Holds the identifier of the key containing the certificate, which shall be verified.
verifyPtr [out]	Holds a pointer to the memory location which will contain the result of the certificate verification.
Return code	
Std_ReturnType	E_OK Request successful.
	E_NOT_OK Request failed.
	CRYPTO_E_BUSY Request failed, Crypto Driver Object is busy.
Functional Description	
Certificate verification.	
Particularities and Limitations	
none	

Verifies the certificate stored in the key referenced by verifyCryptoKeyId with the certificate stored in the key referenced by cryIfKeyId.
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-18 CryIf_CertificateVerify

5.2 Services used by CRYIF

In the following table services provided by other components, which are used by the CRYIF are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError

Table 5-19 Services used by the CRYIF

5.3 Callback Functions

This chapter describes the callback function that is implemented by the CRYIF and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `CryIf_Cbk.h` by the CRYIF.

5.3.1 CryIf_CallbackNotification

Prototype	
`void` **`CryIf_CallbackNotification`** `( Crypto_JobType *job, Std_ReturnType result )`	
Parameter	
`job`	Points to the completed job's information structure. It contains a callbackID to identify which job is finished.
`result`	Contains the result of the cryptographic operation.
Return code	
`void`	none
Functional Description	
Notifies the CRYIF about the completion of the request with the result of the cryptographic operation.	
Particularities and Limitations	
> This function is synchronous. > This function is non-reentrant.	

Table 5-20 CryIf_CallbackNotification

6 Configuration

In the CRYIF the attributes can be configured with the following tools:

> Configuration in DaVinci Configurator 5 Pro for a detailed description see 6.2

6.1 Configuration Variants

The CRYIF supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the CRYIF parameters depend on the supported configuration variants. For their definitions please see the CryIf_bswmd.arxml file.

6.2 Configuration with DaVinci Configurator 5 Pro

6.2.1 General Properties

Attribute Name	Values Default value is typed bold	Description
CryIfVersionInfoApi	TRUE **FALSE**	Pre-processor switch to enable and disable availability of the API Crypto_GetVersionInfo(). - True: API CryIf_GetVersionInfo() is available - False: API CryIf_GetVersionInfo() is not available.
CryIfDevErrorDetect	**TRUE** FALSE	Switches the development error detection and notification on or off. - True: detection and notification is enabled. - False: detection and notification is disabled.
CryIfMaxNumberOfKeyElements	**10**	Size of the maximal amount of element within a key type of all referenced Crypto Drivers
CryIfMaxSizeOfKeyElement	**512**	Size of the largest key element of all referenced Crypto Drivers
CryIfRedirection	TRUE **FALSE**	If enabled, the CryIf supports the redirection feature of ASR 4.4. It is mandatory that the used Csm is also capable of this feature. If disabled, the CryIf ignores any kind of redirection.

Table 6-1 General Properties

6.2.2 Channel Properties

Attribute Name	Values Default value is typed bold	Description
CryIfChannelId		Identifier of the crypto channel
CryIfDriverObjectRef		Reference to a Crypto Driver Object

Table 6-2 Channel Properties

6.2.3 Key Properties

Attribute Name	Values Default value is typed bold	Description
CryIfKeyId		Identifier of the key.
CryIfKeyRef		This parameter refers to a crypto driver key.

Table 6-3 Key Properties

6.2.4 Crypto Module

Attribute Name	Values Default value is typed bold	Description
CryIfSupportsKeyElementCopyPartial	TRUE **FALSE**	If enabled, the referenced driver is capable of the Service Crypto_KeyElementCopyPartial. This API is available since ASR 4.4.x.
CryIfCryptoIncludeFile		Include file of the Crypto module. If CryIfCryptoIncludeFile is not existent, the following pattern is applied for inclusion: Crypto_{Vendor Id}_{Vendor Api Infix}.h
CryIfCryptoMip		Module implementation prefix for the referenced Crypto Module. If not set, then the Mip is determined from BSW-Configuration in the following way: Crypto_{Vendor Id}_{Vendor Api Infix}
CryIfCryptoKeySetValidAPI		The Crypto Api Crypto_KeyValidSet was renamed in ASR 4.3.1 to Crypto_KeySetValid. In order to ensure compatibility, this parameter enables renaming the API call to the corresponding Crypto driver. If this parameter is not present, CryIf will default to KeyValidSet.
CryIfCancelJobApiUsesJobType	TRUE **FALSE**	Specifies whether the referenced driver's Job Cancellation interface uses the Crypto_JobType instead of the Crypto_JobInfoType data type. Set to TRUE for compatibility with ASR R19-11 and the MICROSAR crypto driver. Set to FALSE for compatibility with third-party crypto drivers implementing ASR 4.4 and older. **Attention**: There is an inconsistency in the ASR specs 4.4 and older which, by default, makes it impossible to implement this interface in standard-compliant way. If you set this switch to FALSE, you also need to set the parameter /MICROSAR/Csm/CsmGeneral/CsmEnableVarMembersOfJobType to TRUE. This circumvents the limitations of the ASR spec and makes it possible to provide a compliant interface. Note that this Csm switch only exists in MICROSAR Csm 4.2.0 and above.

Attribute Name	Values Default value is typed bold	Description
CryIfCryptoModuleRef		Reference to the Crypto Driver

Table 6-4 Crypto Module

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
CSM	Crypto Service Manager
CRYIF	Crypto Interface
CRYPTO	Crypto Driver

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com